UNIVERSITY OF JOS

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

EEE 552

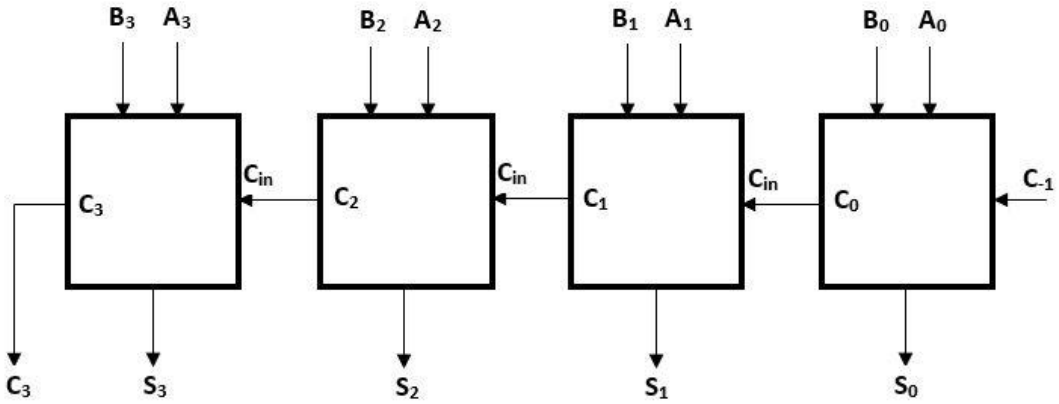
DIGITAL COMPUTATION

I. A. AKINTUNDE

CARRY LOOK AHEAD ADDER

- To overcome the time delay associated with the carry propagation delay experienced in the ripple carry adders, which involve the two bits to be added to be available instantly, with each adder block waiting for the carry to arrive from previous block. A carry look ahead Adder was developed though at the expense of more complex hardware.

Input			Output	
A	B	C_i	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



$$c_O = A.B + (A \oplus B)C_{in}$$
$$C_o = G + PC_{in}$$
$$C_i = G_i + P_iC_{i-1}$$

The concept of CLA will be discussed extensively in class

Example A=1101, B=1011. SUM A and B using CLA

MULTIPLEXER

- Multiplexer is a combinational logic circuit used to select only one input among several inputs based on selection line.
- It can act as digital switch and is also called data selection
- For a MUX, there can be 2^n input, n selection line and only output.

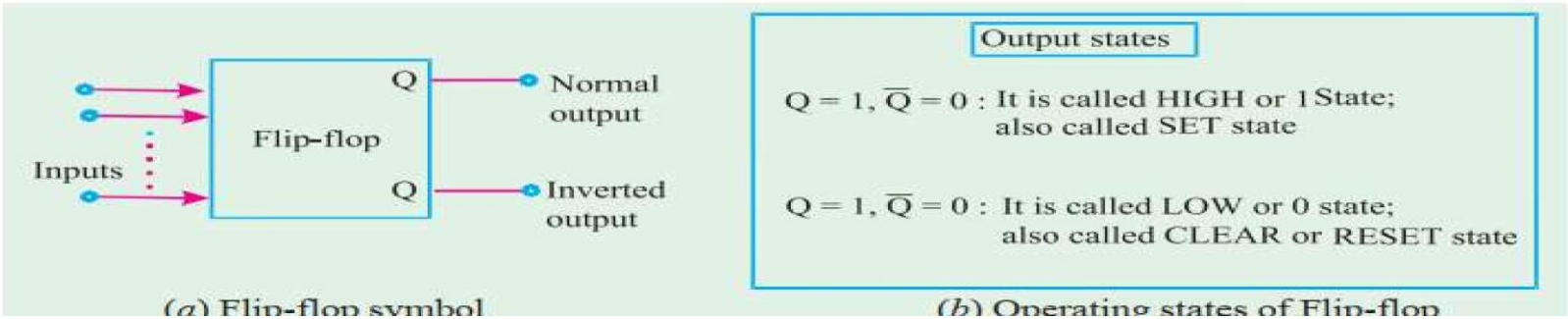
The concept of MUX will be discussed extensively in class

SEQUENTIAL LOGIC CIRCUITS

- Recall: The output logic levels of combinational logic circuits at any instant of time are dependent on the logic levels present at the inputs at that time.
- Any prior input-logic level conditions have no effect on the present outputs. This is due to the fact that combinational logic circuits have no memory.
- Note that: most digital systems are made up of both combinational logic circuits and memory elements.
- The most important memory element is the flip-flop (FF). The FF is made up of an assembly of logic gates.
- Note that: though a logic gate, by itself, has no storage capability, but several such logic gates can be connected together in ways that permit information to be stored.
- Several different gate arrangements are used to produce these flip-flops.
- The flip-flops are used extensively as a memory cell in static random access memory (SRAM) of a computer.
- Examples of such circuits include clocks, flip-flops, bi-stables, counters, memories, and registers. The actions of the circuits depend on the range of basic sub-circuits.

FLIP-FLOPS

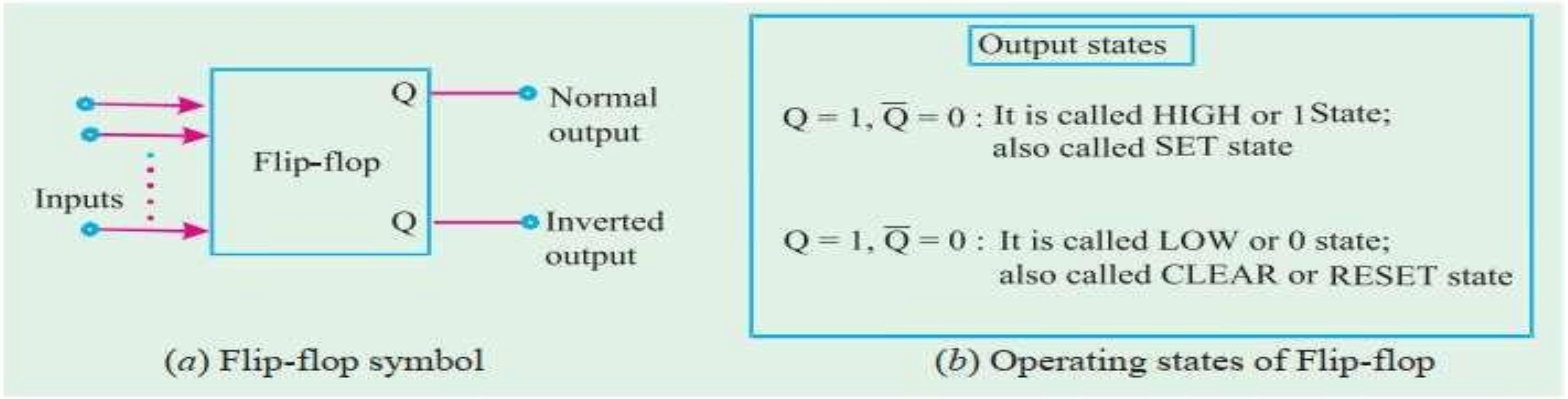
- FF is made up of logic gates and it permits information to be stored. The general symbol used for the representation of flip flop is shown in (a). It has two outputs labelled as Q and ~Q that are inverse (or complement) of each other.
- The Q output is called the normal flip-flop output and ~ Q is the inverted flip-flop output.
- It may be carefully noted that whenever we refer to the state of a flip-flop, we are referring to the state of its normal (Q) output.
- For instance if we say flip-flop is in the HIGH state, we mean that Q = 1. On the other hand if we say flip-flop is in LOW state, we mean that ~Q = 0. It is automatically understood that the ~Q state will always be inverse of Q, i. e., if Q = 1, ~Q = 0, and if Q = 0, ~Q = 1.
- Whenever, the inputs to a FF cause it to go to the Q = 1 state, we call this setting the flipflop or the flip-flop is set.



(a) Flip-flop symbol (b) Operating states of Flip-flop

FLIP-FLOPS

- In a similar way, the state Q = 0, ~Q = 1 is called LOW state or 0 state. It is also called RESET or CLEAR state. Whenever, the inputs to a flip-flop cause it to go to the Q = 0 state, we call this resetting or clearing the flip-flop.
- It may be noted from the flip-flop symbol (Fig. (a)) that a flip-flop can have one or more inputs. These inputs can be used to switch the flip-flop back and forth between its two possible output states.
- Note that a flip-flop is also known as a latch and a bistable multivibrator. The term "latch" is used for certain types of flip-flops that will be described in this class.
- The term bistable multivibrator is the more technical name but flip-flop is the one used more frequently among the engineers and technologists.

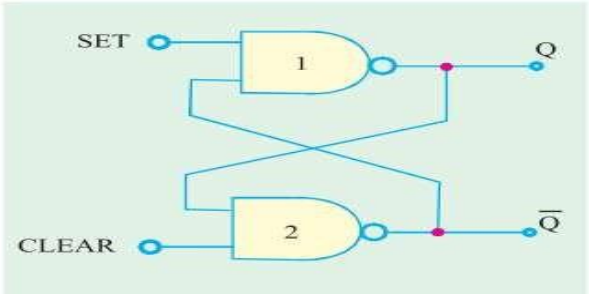


(*a*) Flip-flop symbol

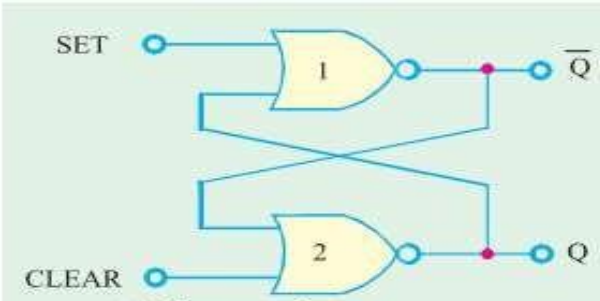
(*b*) Operating states of Flip-flop

LATCH

- A latch is the most basic type of flip-flop circuit. It can be constructed using NAND or NOR gates. Accordingly the latches are of two types :
- NAND gate latch and
- NOR gate latch
- Both these types of latch will be discussed in this class.



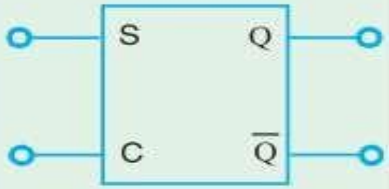
Inputs		Output
SET	CLEAR	
1	1	No change
0	1	$Q = 1$
1	0	$Q = 0$
0	0	$Q = \overline{Q} = 1$ (Invalid)



(a) Construction

Inputs		Output
SET	CLEAR	
0	0	No change
1	0	$Q = 1$
0	1	$Q = 0$
1	1	*Invalid

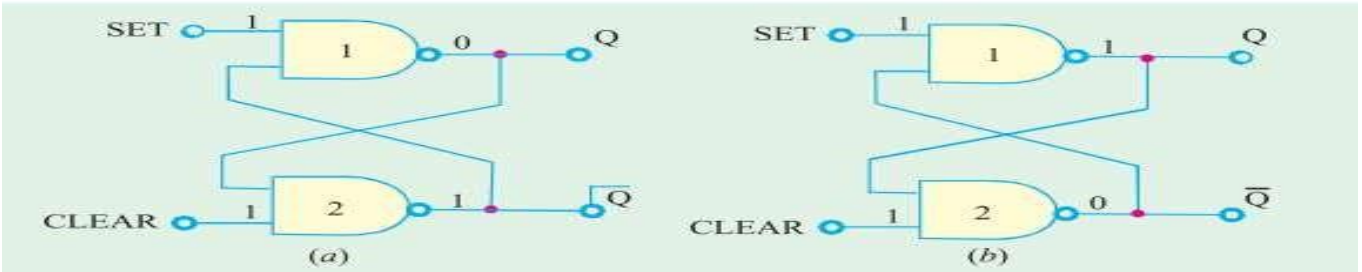
(b) Truth table



(c) Logic symbol

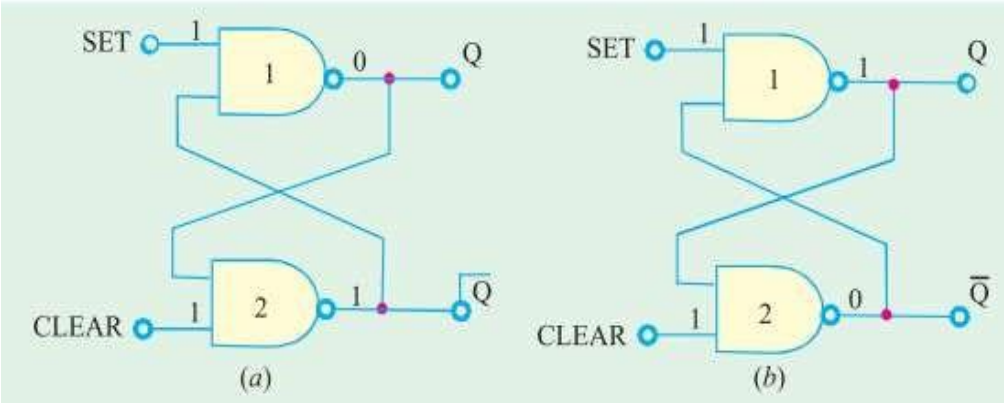
NAND LATCH

- Construction: The NAND latch is shown in the Fig. the two NAND gates are cross-coupled. This means output of NAND-1 is connected to one of the inputs of NAND-2. Similarly, the output of NAND-2 is connected to one of the inputs of NAND-1.
- The NAND gate latch outputs are labelled as Q and ~Q respectively. Under normal conditions, these outputs will always be the inverse (or complement) of each other. The latch has two inputs namely SET and CLEAR.
- The SET input sets Q output of the latch to 1 state while the CLEAR input sets the Q output to the 0 state.
- Operation: The SET and CLEAR inputs of the latch are resting in the HIGH state. Whenever we want to change the latch outputs, one of the two inputs will be pulsed LOW.
- There are two equally likely output states when SET = CLEAR = 1. One possibility is in Fig. (a), Q = 0 and ~Q = 1.



NAND LATCH

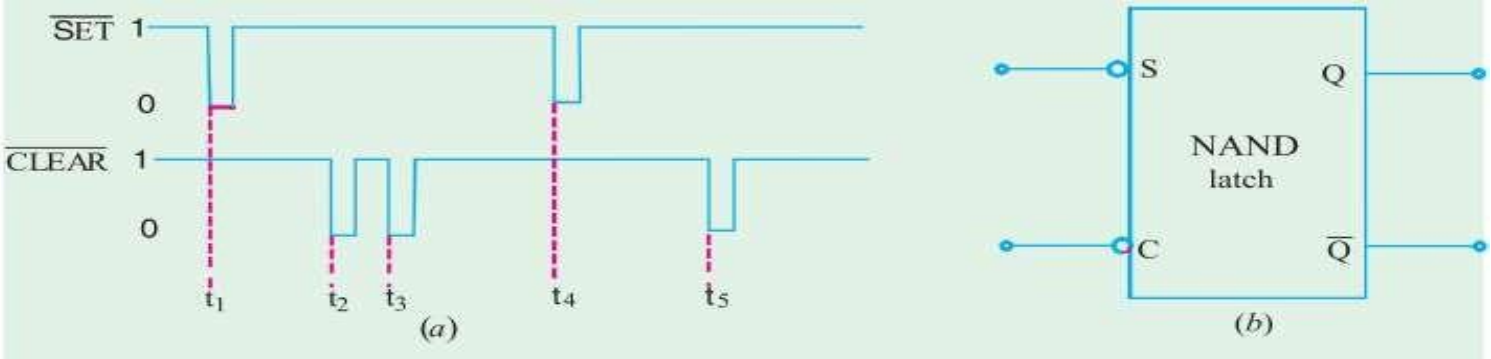
- One possibility is shown in Fig. (Fig a), where we have Q = 0 and ~Q = 1. With Q = 0, the inputs to the NAND-2 are 0 and 1, which produces Q = 1.
- The 1 from ~Q causes NAND-1 to have 1 at both inputs to produce a 0 output at Q. Thus we find that a LOW at the NAND-1 output produces a HIGH at NAND-2 output which in turn keeps the NAND-1 output LOW.
- The second possibility is shown in Fig. (b). Here Q = 1 and ~Q = 0. As seen from this figure, the HIGH from NAND-1 produces a LOW at the NAND-2 output, which in turn keeps the NAND-1 output HIGH. Thus, there are two possible output states when SET = CLEAR = 1. However, which one of these two states exist will depend on what has occurred previously at the inputs.

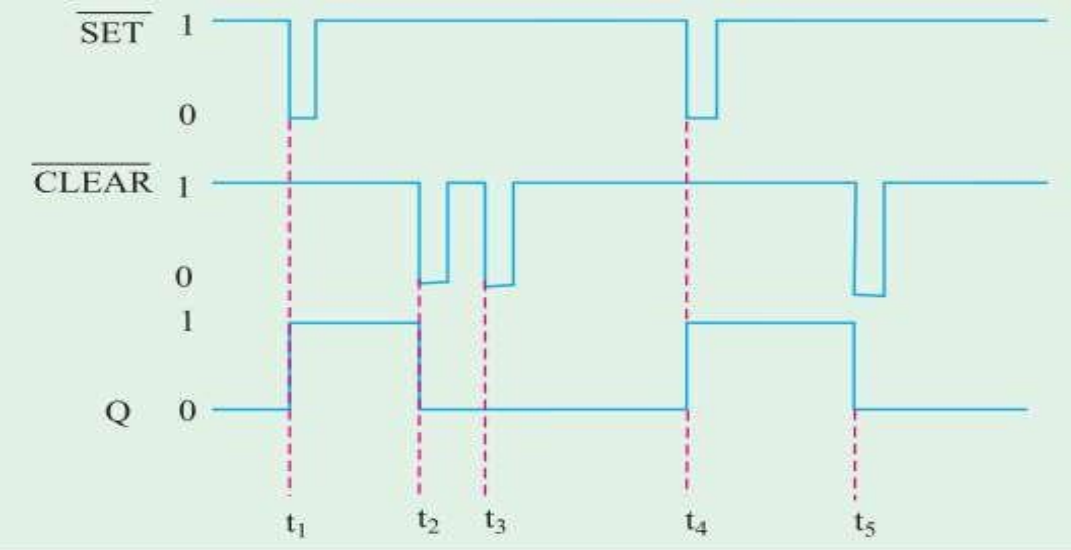


Inputs		Output
SET	CLEAR	
1	1	No change
0	1	$Q = 1$
1	0	$Q = 0$
0	0	$Q = \overline{Q} = 1$ (Invalid)

EXAMPLES USING NAND LATCH

- The waveforms shown in Fig (a) are applied to the inputs of the NAND latch shown in Fig. (b). Assume that initially Q = 0, determine the Q – waveform.



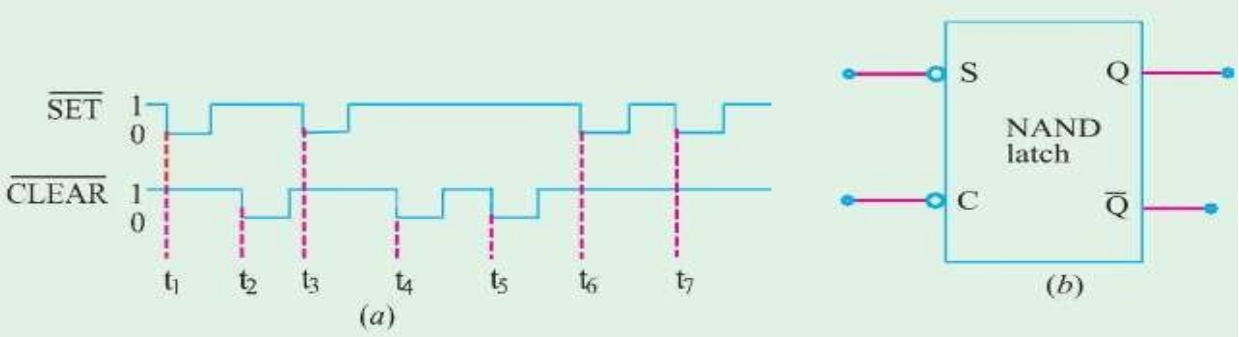


Inputs		Output
SET	CLEAR	
1	1	No change
0	1	$Q=1$
1	0	$Q=0$
0	0	$Q=\bar{Q}=1$ (Invalid)

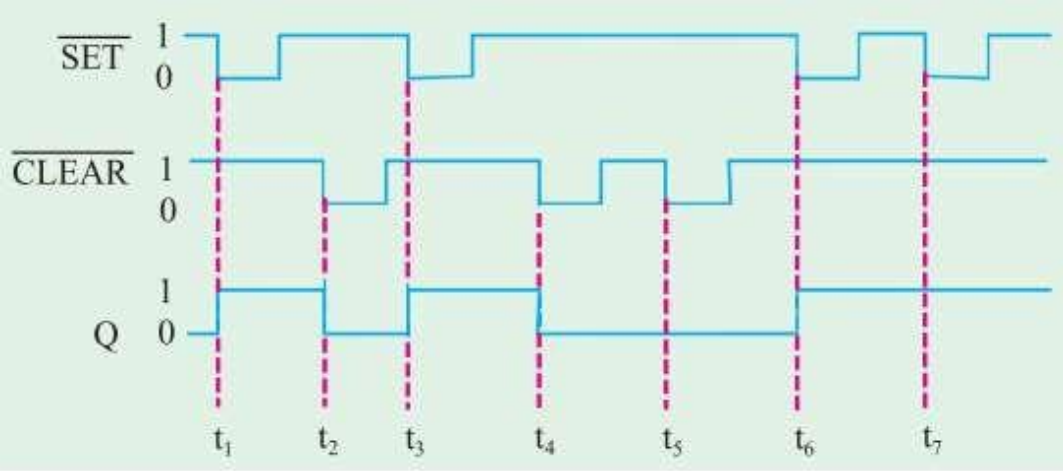
EXAMPLES USING NAND LATCH

- If the SET and CLEAR waveforms shown in Fig (a) are applied to the inputs of NAND latch shown in Fig. (b) determine the waveform that will be observed on the Q output.

  Assume that initially Q = 0.



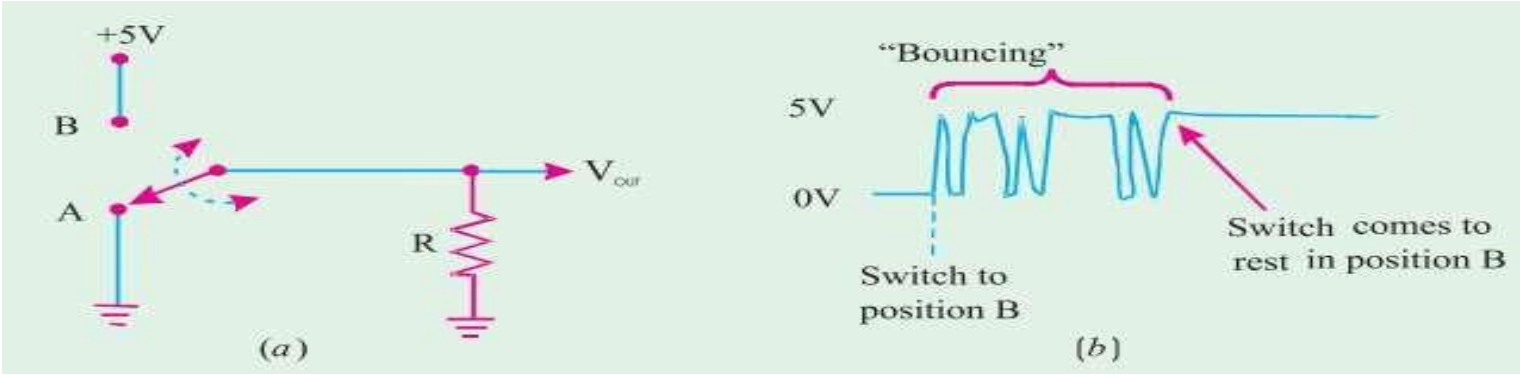
- Solution



Inputs		Output
SET	CLEAR	
1	1	No change
0	1	$Q=1$
1	0	$Q=0$
0	0	$Q=\bar{Q}=1$ (Invalid)

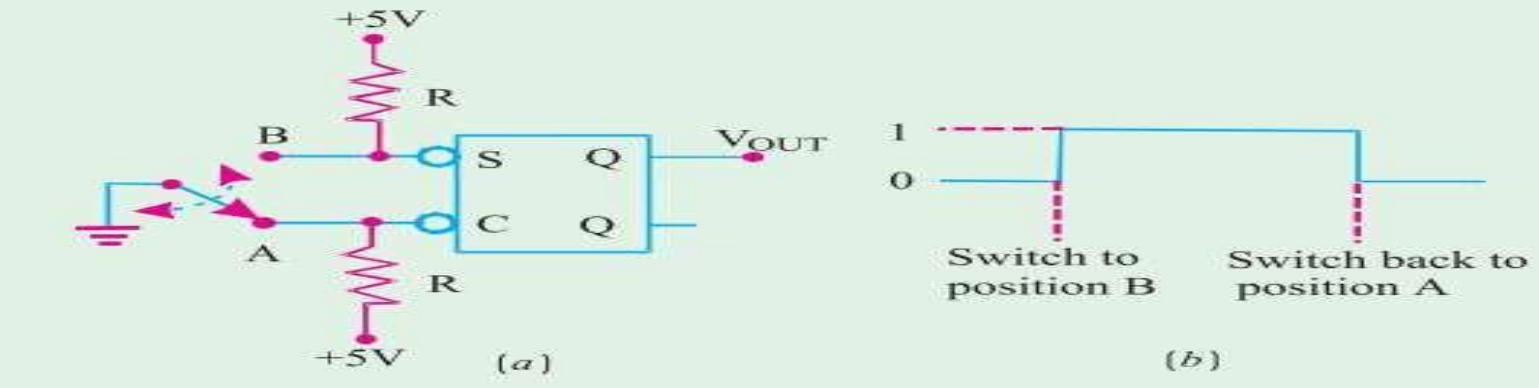
APPLICATION OF NAND LATCH TO DEBOUNCE MECHANICAL SWITCH

- Consider a mechanical switch with contact points A and B connected to Vcc (+ 5V) supply and ground respectively as shown in Fig. (a). It has been found experimentally that when the switch moves from contact position A to B, it produces several output voltage transitions as shown in Fig. (b).
- It is due to a phenomenon called switch bounce i.e. the switch makes and breaks contact with contact B several times before coming to rest on contact B and vice versa.
- Note: the multiple transitions on the output signal generally last only for few milliseconds. But such transitions are unacceptable in many applications. A NAND latch can be used as shown in next slide to eliminate the switch bounce.



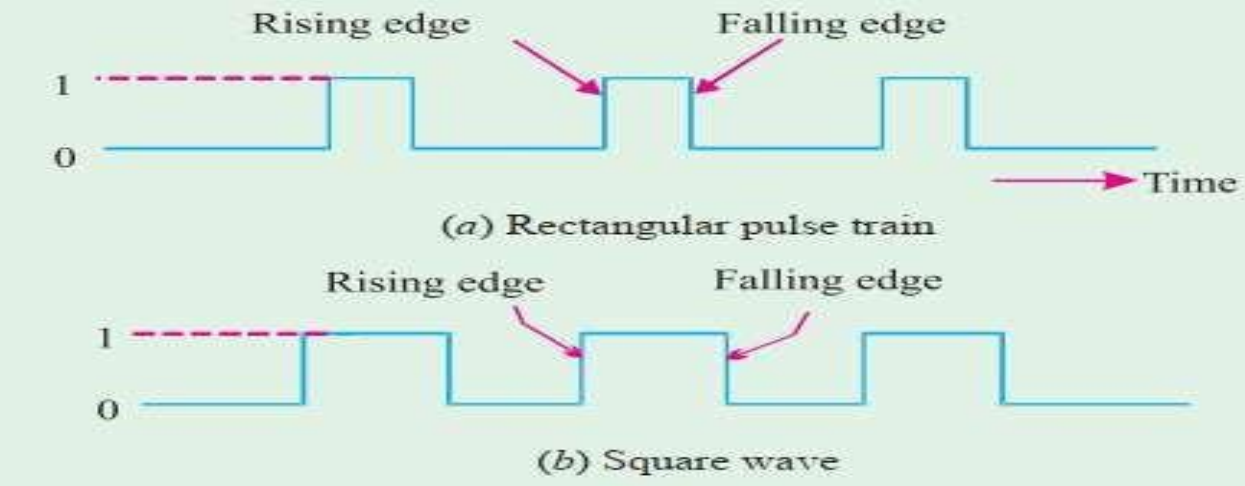
APPLICATION OF NAND LATCH TO DEBOUNCE MECHANICAL SWITCH

- Assume that initially the switch is resting in position A so that CLEAR input is LOW and Q = 0. When the switch is moved to position B, CLEAR will go HIGH and a LOW will appear on the SET input as the switch first makes contact. This will set Q = 1 within few nanoseconds.
- If the switch bounces off contact B, SET and CLEAR will both be HIGH and Q will not be affected, i.e. it will stay HIGH. Thus, nothing will happen at Q as the switch bounces on and off contact B before finally coming to rest.
- Similarly, when the switch is moved from position B back to position A, it will place a LOW on the CLEAR input as it first makes contact. This clears Q to the LOW state.



CLOCKED SIGNALS

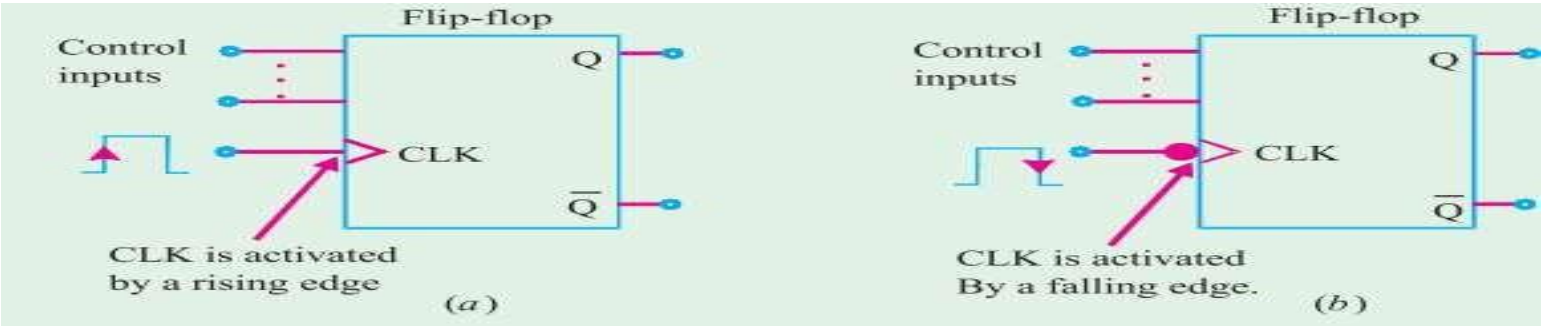
- Digital systems are of two types : Asynchronous systems and the Synchronous systems
- Asynchronous systems: In these systems, the outputs of logic circuits can change state any time one or more of the inputs change. It is more difficult to design and troubleshoot than a synchronous system.
- Synchronous systems: In these systems, the exact time at which the output can change states are determined by a signal commonly called a clock. The clock signal is generally a rectangular pulse train as shown in Fig. (a) or a square wave as shown in Fig. (b). The clock signal is distributed to all parts of the digital system and most of the system outputs can change state only when the clock makes a transition. The transitions are more commonly referred to as edges and are pointed out in the Fig.



CLOCKED SIGNALS

Note: Most digital systems are principally synchronous (although there are always some asynchronous parts). It is because synchronous circuits are easier to design and troubleshoot.

- The synchronization is accomplished through the use of clocked flip-flops that are designed to change states on one or the other of the clock's transitions.

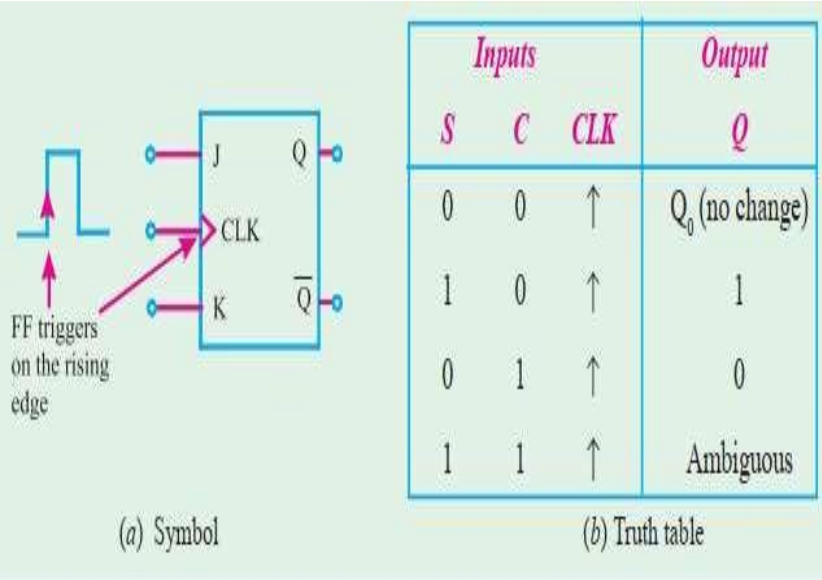


- Control Inputs. Clocked flip-flops usually have one or more control inputs that can have various names depending on their operation.
- The control inputs will have no effect on Q until the active clock transition occurs. In other words, their effect is synchronized with the signal applied to CLK. Because of this reason, the control inputs are referred to as synchronized control inputs.

CLOCKED SET-RESET(CLEAR) FLIP-FLOP

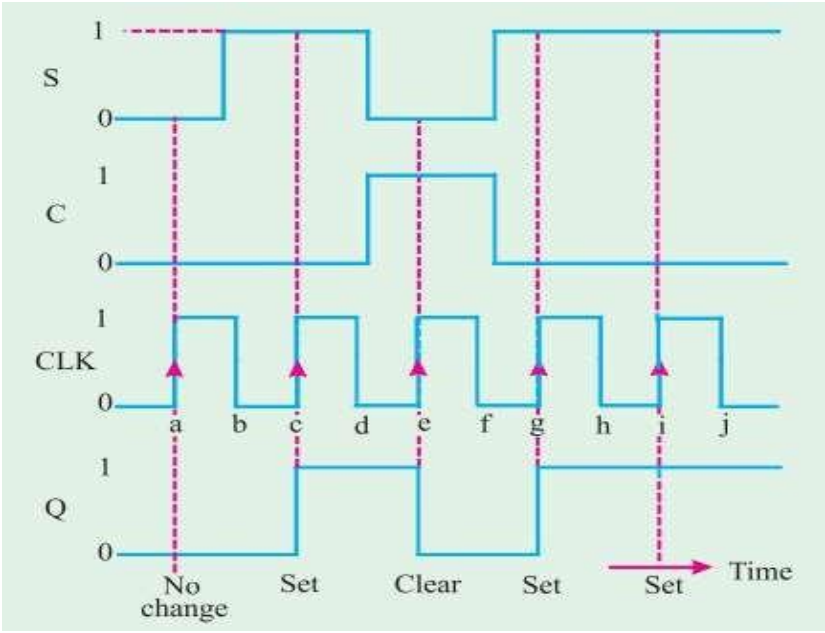
Fig. (a) shows the logic symbol for a clocked S - C flip-flop that is triggered by the rising edge of the clock signal.

- The flip-flop can change output states only when a signal applied to its clock input makes a transition from 0 to 1.
- The S and C inputs control the state of the flip-flop. But it may be noted that the flipflop does not respond to these inputs until the occurrence of the rising edge of the clock signals.



Inputs			Output
S	C	CLK	Q
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	Ambiguous

(b) Truth table

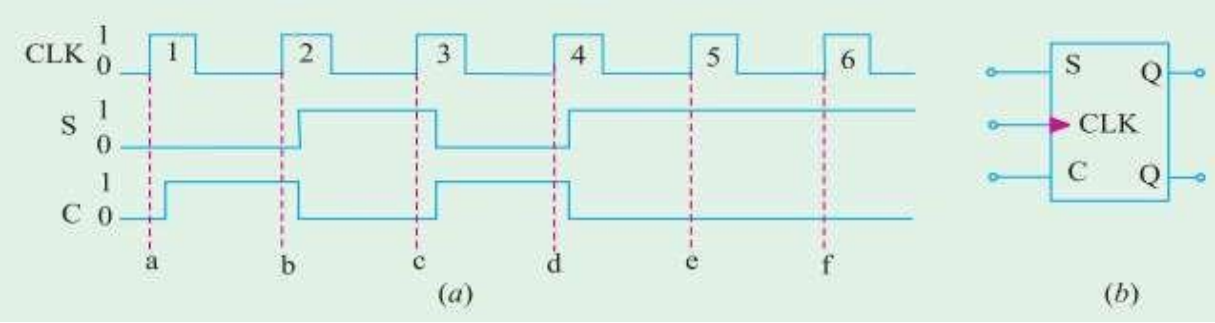


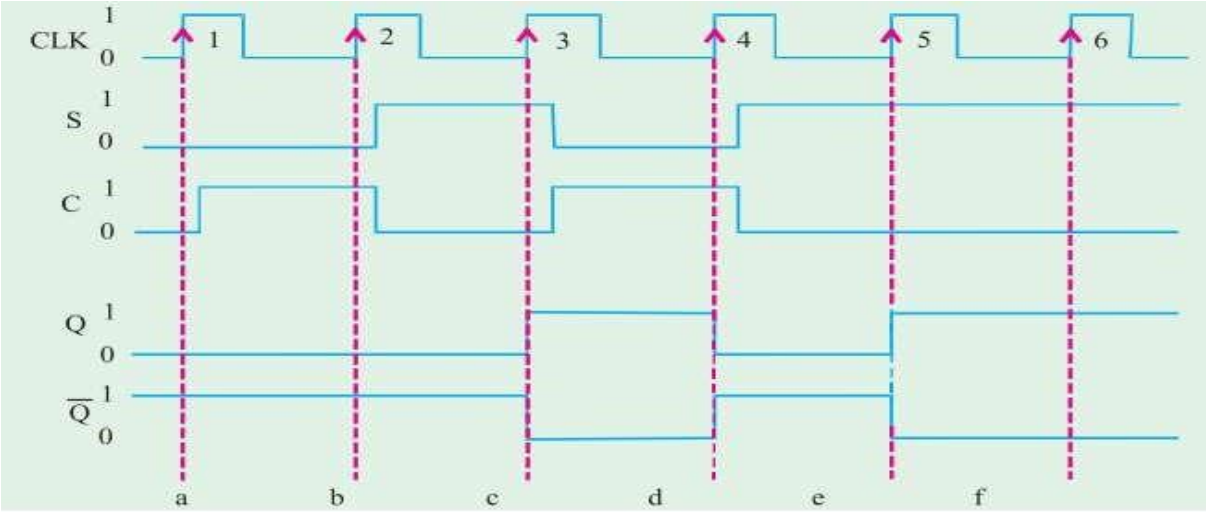
Operation:

1. Initially all the inputs (S, C and CLK) are 0 and the Q output is assumed to be zero, i.e. $Q\_0 = 0$.
2. When the rising edge of the first clock pulse occurs (refer to point 'a'), both the S and C' inputs are 0, so the flip-flop output is not affected and remains in the Q = 0 state (ie. Q = Q_0 =0).
3. At the occurrence of the rising edge of the second clock pulse (refer to point c'), the S input is now HIGH, with C input still LOW.
4. At the occurrence of the rising edge of the third clock pulse (refer to point 'e'), the S' input is LOW with C input HIGH. This causes the flip-flop output to clear to 0 state.
5. The rising edge of the fourth clock pulse sets the flip-flop output to the Q = 1 state (refer to point 'g') because S = 1 and C = 0.
6. The fifth clock pulse also finds that S = 1 and C = 0 at the rising edge (refer to point 'ï'). This situation will produce HIGH output. But since Q is already HIGH, so it remains in that state. This causes the flip-flop output to set to 1 state.

CLOCKED SET-RESET(CLEAR) FLIP-FLOP

Example: Fig. (a) shows the SET and CLEAR waveforms applied at the inputs of the clocked S-C flip-flop shown in Fig. (b). Sketch the waveforms at Q and ~Q outputs of the flip-flop. Assume that initially Q = 0 and ~Q = 1.



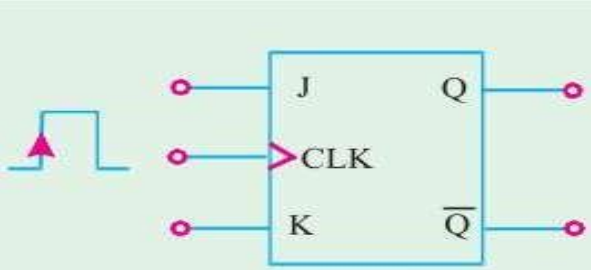


At point 'a', S = C = 0, Q_0 = 0. At point 'b', S = 0 and C = 1, Q remains 0. At point 'c', S = 1 and C = 0, Q = 1. At point 'd', S = 0 and C = 1, Q = 0. At point 'e', S = 1, C = 0, Q = 1. At point 'f', S and C inputs remain the same, therefore Q also remains 1. The sketch of Q-output along with the CLK, S and C waveforms is as shown in the Fig. The ~Q -output waveform is determined by inverting the Q-waveform

Inputs			*Output*
S	*C*	*CLK*	*Q*
0	0	↓	Q_0 (no change)
1	0	↓	1
0	1	↓	0
1	1	↓	Ambiguous

CLOCKED J-K FLIP-FLOP

- Fig. (a) shows the symbol and (b) the truth table for a clocked J-K flip-flop that is triggered by the rising edge of the clock signal. The J and K inputs control the state of the flip-flop in the same manner as the S and C inputs do for the S-C flip-flop.
- However there is one major difference-the J = K = 1 condition in J-K flip-flop does not result in an ambiguous output unlike S = C = 1 in S-C flip-flop for, J = K = 1 condition, the J-K flip-flop will always go to its opposite state upon the rising edge of the clock signal.
- This is called trigger mode. In this mode, if both J and K are left HIGH, the flip-flop will change states (i.e. toggle) for each rising edge of the clock.
- Notice that the truth table is the same except for J = K = 1 condition. This condition results in Q = Q_0which means that the new value of Q will be the inverse of the value it had prior to the rising edge of the clock pulse. It is called toggle operation.
  - To understand the operation of J-K flip-flop, consider the J, K and CLK waveforms shown in the Fig.

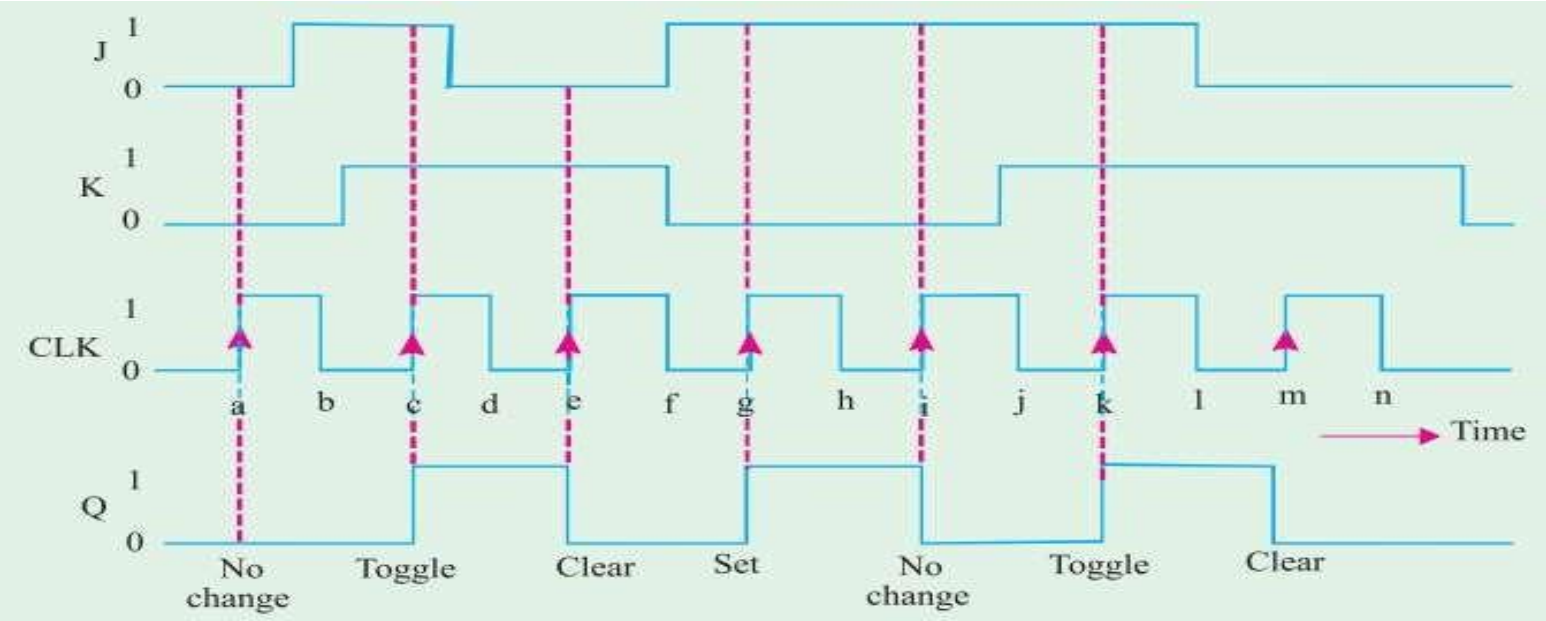


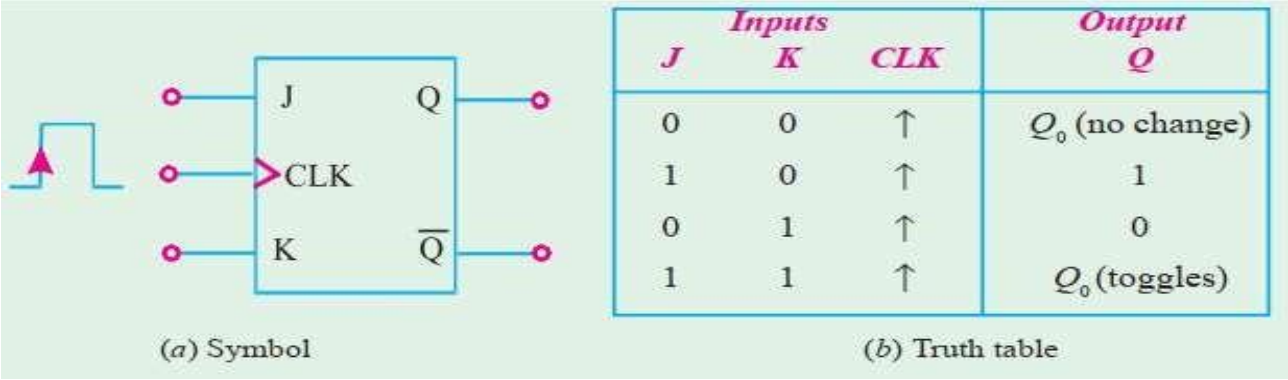
(a) Symbol

Inputs J	Inputs K	Inputs CLK	Output Q
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	Q_0 (toggles)

(b) Truth table

CLOCKED J-K FLIP-FLOP





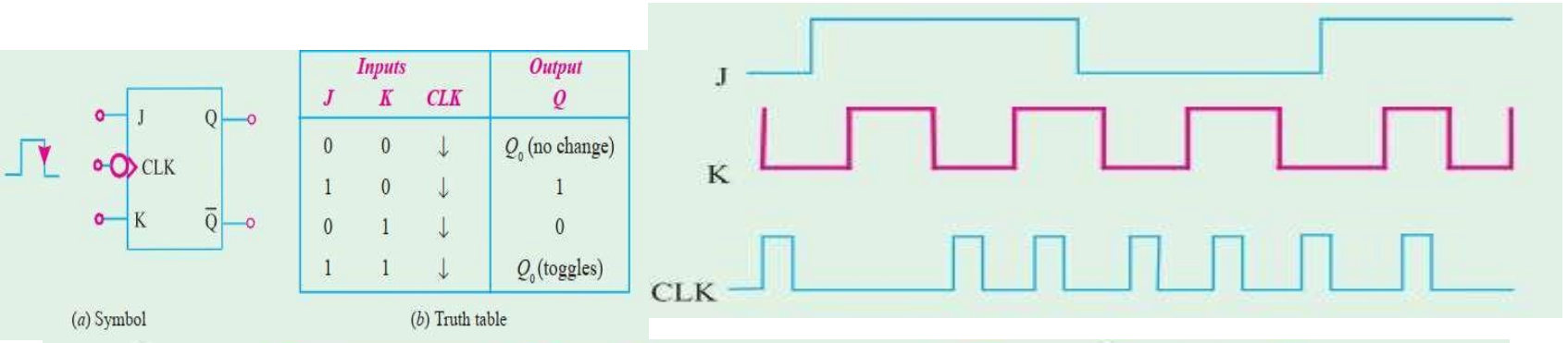
(*a*) Symbol

Inputs			*Output*
J	*K*	*CLK*	*Q*
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	Q_0 (toggles)

(*b*) Truth table

CLOCKED J-K FLIP-FLOP

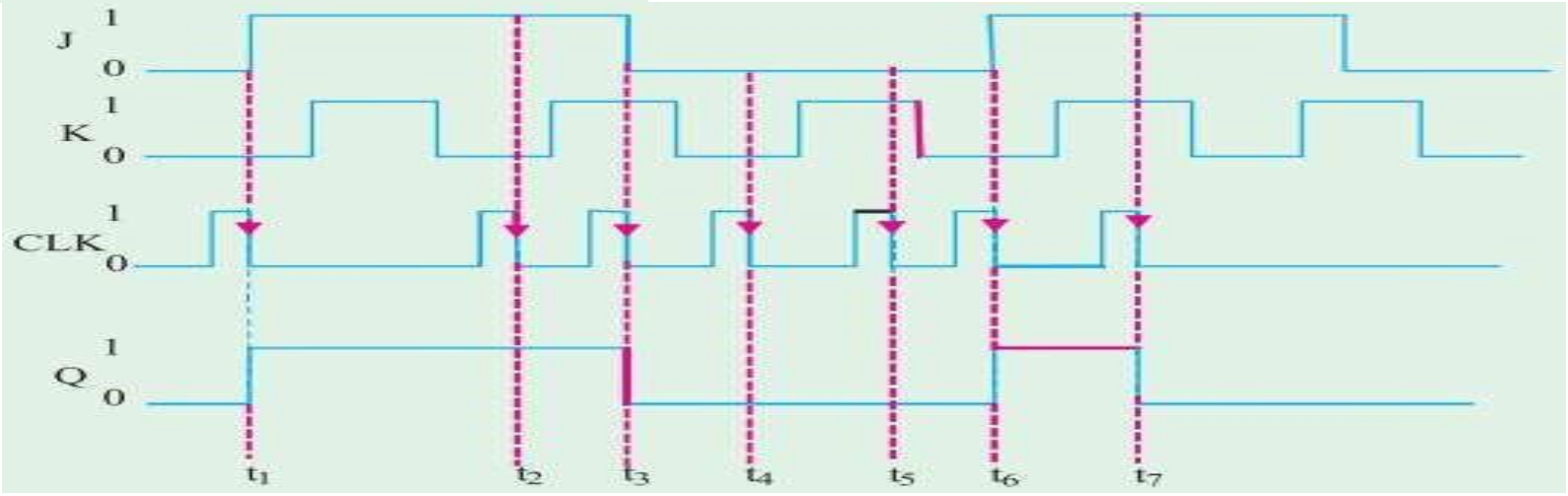
- Example: What will be the output waveform Q of a J-K flip-flop if the following waveforms are applied at the input? Assume the flip-flop triggers at the falling edge of clock pulse



(a) Symbol

Inputs			Output
J	K	CLK	Q
0	0	↓	Q_0 (no change)
1	0	↓	1
0	1	↓	0
1	1	↓	Q_0 (toggles)

(b) Truth table



INTEGRATED CIRCUIT

- The invention of the transistor in 1948 by W.H. Brattain and I. Bardeen, led to considerable reduction in size of electronic circuits.
- It was because a transistor is cheaper, more reliable, less power consuming and much smaller in size than an electron tube.
- To take advantage of small transistor size, the passive components (resistor, capacitors and inductors) too were greatly reduced in size thereby making the entire circuit very small.
- Development of printed circuit boards (PCBs) further reduced the size of electronic equipment by eliminating bulky wiring and tie points.
- In the early 1960s, a new field of microelectronics was born primarily to meet the requirements of the Military which wanted to reduce the size of its electronic equipment to approximately one tenth of its then existing volume.

INTEGRATED CIRCUIT

- This drive for extreme reduction in the size of electronic circuits has led to the development of microelectronic circuits called integrated circuits (ICs)
- This ICs are so small that their actual construction is done by technicians using high powered microscopes.
- An IC is a complete electronic circuit in which both the active and passive components are fabricated on a tiny single chip of silicon.
- Active components are those which have the ability to produce gain. Examples are: transistors and FETs.
- Passive components or devices are those which do not have this ability. Examples are: resistors, capacitors and inductors.
- ICs are produced by the same processes as are used for manufacturing individual transistors and diodes etc.

ADVANTAGES OF INTEGRATED CIRCUIT

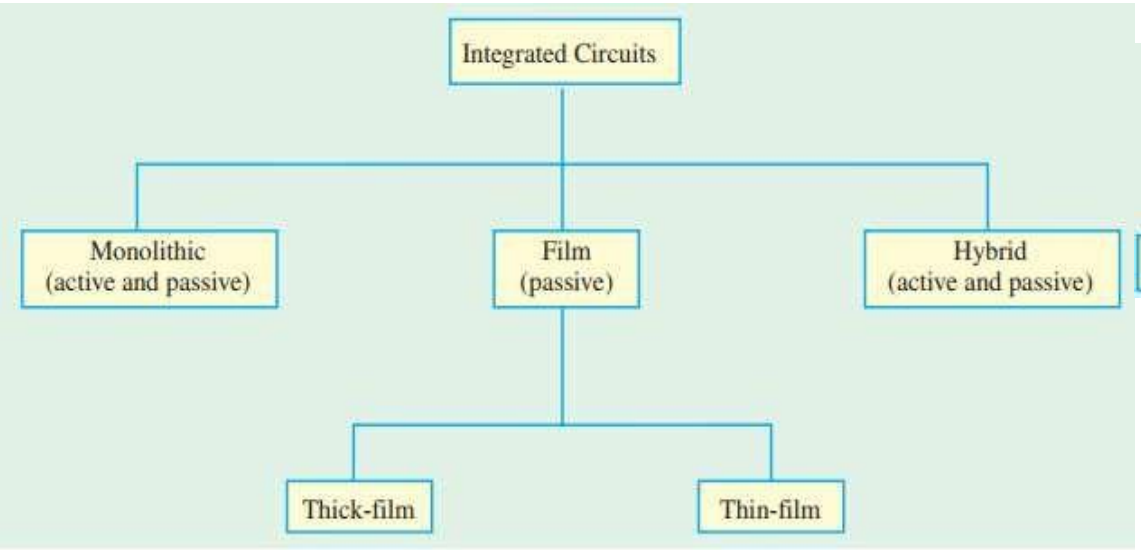
- Extremely small physical size: The various components and their interconnections are distinguishable only under a powerful microscope
- Very small weight: Since many circuit functions can be packed into a small space, complex electronic equipment can be employed in many applications where weight and space are critical, such as in aircraft or space-vehicles.
- Reduced cost: It is a major advantage of ICs. The reduction in cost per unit is because many identical circuits can be built simultaneously on a single wafer.
- Reduced cost: It is a major advantage of ICs. The reduction in cost per unit is because many identical circuits can be built simultaneously on a single wafer—this process is called batch fabrication. Although the processing steps for the wafer are complex and expensive, the large number of resulting integrated circuits make the ultimate cost of each IC fairly low.
- Extremely high reliability: It has high reliability due to the absence of soldered connections, require fewer interconnections, which is the major cause of circuit failures. Small temperature rise due to low power consumptions of ICs also improves their reliability.

-
- Increased response time and speed: Since various components of an IC are located close to each other in or on a silicon wafer, the time delay of signals is reduced. Moreover, because of the short distances, the chance of stray electrical pickup (called parasitic capacitance) is practically nil. Hence it makes them very suitable for small signal operation and high frequency operation
- Low power consumption: Because of their small size, ICs are more suitable for low power operation than bulky discrete circuits.
- Easy replacement: ICs are hardly ever repaired because in case of failure, it is more economical to replace them than to repair them.
- Higher yield: The yield is the percentage of usable devices. Because of the batch fabrication, the yield is very high. Faulty devices usually occur because of some defect in the silicon wafer or in the fabrication steps.

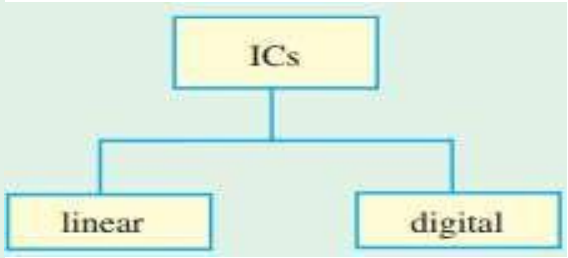
DRWBACKS OF INTEGRATED CIRCUIT

- Coils or inductors cannot be fabricated
- ICs function at low voltages
- They handle only limited amount of power,
- They are quite delicate and cannot withstand rough handling or excessive heat.

Note: the advantages of ICs far outweigh their disadvantages or drawbacks.



Classification of ICs By Structure



Classification of ICs By Function

1. OR Laws

These four laws have already been discussed in the previous chapter. These are

Law 1. $A+0=A$ Law 3. $A+A=A$

Law 2. $A+1=1$ Law 4. $A+\overline{A}=1$

2. AND Laws

Law 5. $A.0=0$ Law 7. $A.A=A$

Law 6. $A.1=A$ Law 8. $A.\overline{A}=0$

Fig. 1

3. Laws of Complementation

Law 9. $\overline{0}=1$ Law 10. $\overline{1}=0$

Law 11. if $A=0$, then $\overline{A}=1$ Law 12. if $A=1$, then $\overline{A}=0$

Law 13. $\overline{\overline{A}}=A$

4. Commutative Laws

These laws allow ***change in the position*** of variables in OR and AND expressions.

Law 14. $A+B=B+A$ Law 15. $A.B=B.A$

5. Associative Laws

These laws allow removal of brackets from logical expression and regrouping of variables.

Law 16. $A+(B+C)=(A+B)+C$

Law 17. $(A+B)+(C+D)=A+B+C+D$ Law 18. $A.(B.C)=(A.B).C$

6. Distributive Laws

These laws permit factoring or multiplying out of an expression.

Law 19. $A(B+C)=AB+AC$ Law 20. $A+BC=(A+B)(A+C)$

Law 21. $A+\overline{A}.B=A+B$

7. Absorptive Laws

These enable us to reduce a complicated logic expression to a simpler form by absorbing some of the terms into existing terms.

Law 22. $A+AB=A$ Law 23. $A.(A+B)=A$

Law 24. $A.(\overline{A}+B)=AB$

LAWS OF BOOLEAN ALGEBRA

Example: Prove the following Boolean identity : AC + ABC = AC

Solution: Taking the left hand side expression as y, we get

- y = AC + ABC= AC (1 + B) Now 1 + B = 1 (OR Law 2) ∴y=AC.1= AC (Law 6) ∴ AC+ABC=AC
- Example: Determine the logic expression for the output Y, from the truth table shown in the Fig. Simplify and sketch the logic circuit for the simplified expression.

Inputs			Output
A	B	C	Y
0	0	0	0
0	0	1	1 → $\bar{A}\ \bar{B}\ C$
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1 → $A\ \bar{B}\ C$
1	1	0	0
1	1	1	0

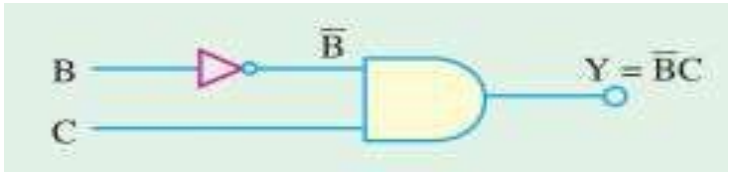
The resulting expression for the output,

$Y = \bar{A}\ \bar{B}\ C + A\ \bar{B}\ C$ ...(Law 19)

$= (\bar{A} + A)\ \bar{B}\ C$ ...(Law 4)

$= 1.\ \bar{B}\ C$ ...(Law 6)

$= \bar{B}\ C$



Example: Prove the following Boolean identity : (A + B) (A + C) = A + BC

Y = (A + B)(A + C) = AA+AC+AB+BC

= A+ AC + AB + BC = A + AB + AC + BC

= A (1 + B) + AC + BC = A + AC + BC

= A (1 + C) + BC= A + BC

∴ (A + B) (A + C) = A + BC

. Prove the following identity : $A + \bar{A}B = A + B$

the left-hand side expression be put equal to Y.

$$Y = A + \bar{A}B = A.1 + \bar{A}B \quad \text{—Law 6}$$
$$= A(1+B) + \bar{A}B \quad \text{—Law 12}$$
$$= A.1 + AB + \bar{A}B \quad \text{—Law 19}$$
$$= A + BA + B\bar{A} \quad \text{—Law 6 and 15}$$
$$= A + B(A + \bar{A}) \quad \text{—Law 19}$$
$$= A + B.1 \quad \text{—Law 4}$$
$$= A + B \quad \text{—Law 6}$$
$$A + \bar{A}B = A + B$$

Prove the following Boolean identity : $(A+B)(A+\bar{B})(\bar{A}+C) = AC$

(Digital Computations, Punjab Univ. 1990)

the left-hand side expressionbe represented by Y.

$$Y = (A+B)(A+\bar{B})(\bar{A}+C) = (AA + A\bar{B} + BA + B\bar{B})(\bar{A}+C)$$
$$= (A + AB + A\bar{B})(\bar{A}+C) = [A(1+B) + A\bar{B}](\bar{A}+C) \quad \because B\bar{B} = 0$$
$$= (A + A\bar{B})(\bar{A}+C) = A(1+\bar{B})(\bar{A}+C)$$
$$= (A.1)(\bar{A}+C) = A(\bar{A}+C) = A\bar{A} + AC = AC \quad \because A\bar{A} = 0$$

Example 71.8. *Simplify the following expression and show the minimum gate implementation.*

$$Y = A.B.\bar{C}.\bar{D} + \bar{A}.B\bar{C}.\bar{D} + B.\bar{C}.D$$

Solution. As seen from *OR* and *AND* laws of Art 67.3, $A + \bar{A} = 1$ and $A.1 = A$

$$\therefore \quad Y = B\bar{C}\bar{D}(A+\bar{A}) + B\bar{C}D = B\bar{C}\bar{D}.1 + B\bar{C}D.$$
$$= B.\bar{C}.\bar{D} + B.\bar{C}.D = B.\bar{C}.(D+\bar{D}) = B.\bar{C}.1 = B.\bar{C}.$$

Minimum gate implementation is shown by the circuit of Fig. 71.7

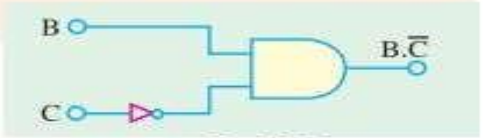


Fig. 71.7

DE MORGAN'S THEOREM

- These two theorems (or rules) are a great help in simplifying complicated logical expressions. The theorems can be started as under:

Law 25. $\overline{A+B} = \bar{A} \cdot \bar{B}$ Law 26. $\overline{A \cdot B} = \bar{A} + \bar{B}$

- The first statement says that the complement of a sum equals the product of complements. The second statement says that the complement of a product equals the sum of the complements. In fact, it allows transformation from a sum-of-products from to a product-of-sum from. The circuits in the fig. illustrate De Morgan's theorems.

- As seen from the above two laws, the procedure required for taking out an expression from under a NOT sign is as follo

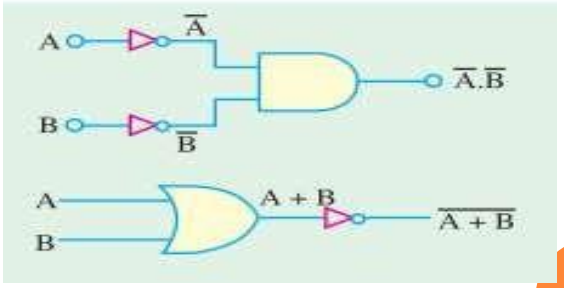
1. *complement the given expression i.e., remove the overall NOT sign*
2. *change all and ANDs to ORs and all the ORs to ANDs.*
3. *complement or negate all individual variables.*

Example:

$$\overline{A+BC} = A+BC \quad \text{—step 1}$$
$$= A(B+C) \quad \text{—step 2}$$
$$= \bar{A}(\bar{B}+\bar{C}) \quad \text{—step 3}$$

Next, consider this example,

$$\overline{(\bar{A}+B+\bar{C})(\bar{A}+B+C)} = (\bar{A}+B+\bar{C})(\bar{A}+B+C) \quad \text{—step 1}$$
$$= \bar{A}B\bar{C} + \bar{A}BC \quad \text{—step 2}$$
$$= \bar{\bar{A}}\bar{B}\bar{\bar{C}} + \bar{\bar{A}}\bar{B}\bar{C} \quad \text{—step 3}$$
$$= A\bar{B}C + A\bar{B}\bar{C}$$



DE MORGN'S THEOREM

Example: Demorganise the expression $\overline{(A+B)(C+D)}$

Solution. The procedure is as explained above.

$$\overline{(A+B)(C+D)} = (A+B)(C+D) \quad \text{—step 1}$$
$$= (AB)+(CD) \quad \text{—step 2}$$
$$= \overline{AB} + \overline{CD} \quad \text{—step 3}$$

Simplify each of the following expression using De Morgans theorems:

(a) $A\overline{(B+\overline{C})D}$

(b) $\overline{(M+\overline{N})(\overline{M}+N)}$ (c) $\overline{\overline{AB}\,\overline{C}\,D}$

Solution. Please note that there is more than one way of simplifying the expressions given in part (*a*), (*b*) and (*c*).

(*a*)
$$\overline{A\overline{(B+\overline{C})D}} = A\overline{(B+\overline{C})}\,D \quad \text{... step 1}$$
$$= A + \overline{(B+\overline{C})} + D \quad \text{... step 2}$$
$$= \overline{A} + \overline{\overline{B+\overline{C}}} + \overline{D} \quad \text{... step 3}$$
$$= \overline{A} + B + \overline{C} + \overline{D} \quad \text{...}(\because \overline{\overline{B+\overline{C}}} = B+\overline{C})$$

(*b*)
$$\overline{(M+\overline{N})(\overline{M}+N)} = (M+\overline{N})(\overline{M}+N) \quad \text{... step 1}$$
$$= (M\overline{N}) + (\overline{M}N) \quad \text{... step 2}$$
$$= (\overline{M}\,\overline{\overline{N}}) + \overline{\overline{M}}\,\overline{N} \quad \text{... step 3}$$
$$= \overline{M}N + M\overline{N} \quad \text{... step 4}$$

(*c*)
$$\overline{\overline{AB}\,\overline{C}\,D} = \overline{\overline{AB}\,\overline{C}}\,D \quad \text{... step 1}$$
$$= \overline{\overline{AB}\,\overline{C}} + D \quad \text{... step 2}$$
$$= \overline{\overline{\overline{AB}\,\overline{C}}} + \overline{D} \quad \text{... step 3}$$
$$= \overline{AB}\,\overline{C} + \overline{D} \quad \text{...}(\because \overline{\overline{\overline{AB}\,\overline{C}}} = \overline{AB}\,\overline{C})$$

The term $\overline{AB}\,\overline{C}$ can be simplified further by De Morganising the term $\overline{AB}$ as $\overline{A} + \overline{B}$. Thus

$$\overline{\overline{AB}\,\overline{C}\,D} = \overline{AB}\,\overline{C} + \overline{D} = (\overline{A}+\overline{B})\,\overline{C} + \overline{D} = \overline{A}\,\overline{C} + \overline{B}\,\overline{C} + \overline{D}$$

STANDARD FORMS OF BOOLEAN EXPRESSIONS

- All Boolean expressions regardless of their form, can be converted into either of the two following standard forms:
- Sum-of-products (SOP) form and 2. Product-of-sums (POS) form.
- The standardization of Boolean expressions makes their evaluation, simplification and implementation much more systematic and easier

The Sum-of-Products (SOP) Form

- A product term is a term consisting of the product (or Boolean multiplication) of literals (i.e. variables or their complements). When we add two or more product terms, the resulting expression is called, sum-of-products (SOP) expression. Some examples of sum-of-products
- expressions are AB ABC + , ABC ACD ABCD AB + + , + AC + A B C
- Sometimes, it is convenient to define the set of variables contained in the expression (in either complemented or uncomplemented form) as a domain. For example, the domain of the expression A BC + ABC + A B CD is the set of variables A, B, C and D.
- Any logic expression can be changed into SOP form by applying Boolean algebra laws and theorems. For example, the expression A(BC + D) can be converted to SOP form by applying the distributive law : A (BC + D) = ABC +AD

THE STANDARD SOP FORM

- In some SOP (sum-of-products) expressions, the product terms may not contain all the variables in the domain of the expression. E.g. the expression, ABC+ACD+ABCD has a domain made up of the variables A, B, C and D.
- Note hat the first two terms contains only three variables, i.e. D or not D is missing from the first term and B or not B is missing from the second term.
- A standard SOP expression is defined as an expression in which all the variables in the domain appear in each product term. For example $\bar{A}BCD+A\bar{B}\bar{C}D+A\bar{B}CD$ is a standard SOP expression.
- Standard SOP expressions are important in constructing truth tables and in karnaugh map simplification method. It is very straightforward to convert non-standard product term to a standard SOP using Boolean algebra. Each product term in the SOP expression that does not contain all the variables in the domain has to be expanded to standard form to include all the variables in the domain and their complements.
- As stated below, a nonstandard SOP expression is converted into standard form using Boolean algebra law 4 $A+\bar{A}=1$:i.e. a variable added to its complement equals 1.

THE STANDARD SOP FORM

- Step 1. Multiply each nonstandard product term by a term made up of the sum of a missing variable and its complement. This results in two product terms. It is possible because we know that we can multiply anything by 1 without changing its value.
- Step 2. Repeat step 1 until all resulting product terms contain all variables in the domain in either complemented or uncomplemented form. Note that in converting a product term to a standard form, the number of product terms is doubled for each missing variable.
- For example, suppose we want to convert the Boolean expression $\bar{A}BC+A\bar{C}D+A\bar{B}CD$ to a standard SOP form. Then following the above procedure we proceed as below: The expression for standard SOP is shown

$$\bar{A}BC(D+\bar{D})+A(B+\bar{B})\bar{C}D+A\bar{B}CD$$

$$=\bar{A}BCD+\bar{A}BC\bar{D}+AB\bar{C}D+A\bar{B}\bar{C}D+A\bar{B}CD$$

THE PRODUCT-OF-SUMS (POS) FORM

- The sum term is a term consisting of the sum (or Boolean addition) of literals (i.e. variables or their complements). When we multiply two or more sum terms, the resulting expression is called product of-sums (POS). Some examples of POS form are $(A + \overline{B})(\overline{A} + B + C)$

$(\overline{A} + \overline{B})(A + C)(\overline{A} + \overline{B} + C)$

- It may be carefully noted that a POS expression, can contain a single-variable term as in ~A (A + ~B + C) (B + ~C + ~D). In POS expression, a single overbar cannot extend over more than one variable, although more than one variable in a term can have an overbar.

The Standard POS Form

- Some POS (product-of-sums) expressions in which some of the sum terms do not contain all the variables in the domain of the expression such as the expression, (~ A + B + C), (A + ~C + D) (A + ~B + C + D) has a domain made up of variables, A, B, C and D.
- Notice that the complete set of variables in the domain is not represented in the first two terms of the expression, i.e. D or ~D is missing in the first term and B or ~B is missing in the second term.

THE PRODUCT-OF-SUMS (POS) FORM

- A standard POS expression is defined as an expression in which all the variables in the domain appear in each sum term. For example (A + B + C + D) (A + B + C + D) (A + B + C + D) is a standard POS form.
- Any nonstandard POS expression can be converted to a standard form using Boolean algebra. Each sum term in an POS expression that does not contain all the variables in the domain can be expanded to standard form to include all variables in the domain and their complements.
- As stated below : a nonstandard POS expression is converted into a standard form using Boolean algebra Law 8 : A .~ A = 0, i.e. a variable multiplied by its complemented equals 0.
- Step 1. Add to each nonstandard product term a term made up of the product of a missing variable and its complement. This results in two sum terms. This is possible because we know that we can add 0 to anything without changing its value.
- Step 2. Apply law 20, i.e. A + BC = (A + B) (A + C)
- Step 3. Repeat Step 1 until all resulting sum terms contain all variables in the domain in either complemented or uncomplemented form. For example we want to convert the Boolean expression

$$(\overline{A}+B+C+D\,\overline{D})(A+B\,\overline{B}+\overline{C}+D)(A+\overline{B}+C+D)$$
$$=(\overline{A}+B+C+D)(\overline{A}+B+C+\overline{D})(A+B+\overline{C}+D)(A+\overline{B}+\overline{C}+D)(A+\overline{B}+C+D)$$

PRACTICE QUESTIONS

- A microwave oven requires that the main power supply be on, the door be latched, and the inner set to a time other than zero before the oven will function. Draw a logic diagram for the operation of the oven.
- A lighter is being sold that uses a light beam to the light the lighter. The lighter has a cut out on its side that has been fixed with a light source and a light sensor. Opening the lid of the lighter activates the light source; interrupting the light source causes the lighter to light. Draw a logic diagram for the function of the lighter.
- An automatic parking lot gate extends a ticket to the customer when the customer's car drives over a sensor S and the driver presses button B on the ticket vendor. Draw the logic diagram to control the release of the ticket. Once the extended ticket is taken from dispenser D, the arm A will raise to let the car through. Draw the logic diagram that control the raising of the arm.
- Formulate a question on the sequence of lecture on EEE 552. Draw the equivalent logic circuit for the formulated Question.

USES OF SIMPLE LOGIC

Example

- Heating Boiler; If chimney is not blocked and the house is cold and the pilot light is lit, then open the main fuel valve to start boiler.

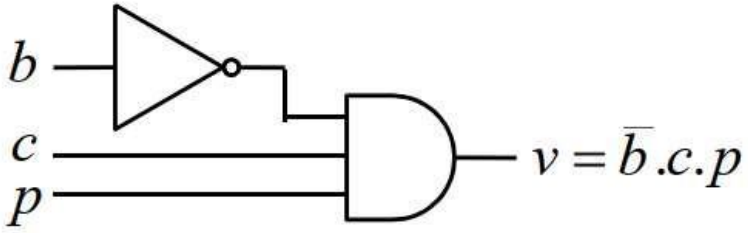
Solution

b = chimney blocked c = house is cold p = pilot light lit v = open fuel valve

So in terms of a logical (Boolean) expression

v = (NOT b) AND c AND p

- Draw the circuit.



ASSIGNMENT

1) An automatic parking lot gate extends a ticket to the customer when the customer's car drives over a sensor S and the driver presses button B on the ticket vendor. Draw the logic diagram to control the release of the ticket. Once the extended ticket is taken from dispenser D, the arm A will raise to let the car through. Draw the logic diagram that control the raising of the arm.

2) Succinctly write a short note on each of the following computational concepts as it relates to digital computation or digital electronics

i. Hazards
ii. Meta-stability
iii. False paths
iv. Inertial delay
v. Sticky bits
vi. Clock skew-jitter
vii. Dynamic and static sensitization

Due Date: 05/08/2025